hhu

hhu
Heinrich Heine
Universität Düsseldorf

Betriebssysteme und Systemprogrammierung

4. Rust (Part 2)

Michael Schöttner

4.1 Outline

Heinrich Heine
Universität Düsseldorf

- Basics
- Advanced stuff
- Unsafe stuff

4.2 Advanced stuff

Heinrich Heine
Universität Düsseldorf

- Memory regions
- Traits & Generics
- Printing structs
- Option & Error handling
- Modules & cargo
- Slices
- String vs str
- Life times
- Unsafe

Memory regions

Heinrich Heine
Universität Düsseldorf

- Data memory
  - For data that is fixed in size and static (i.e. always available through life of program).
  - Compilers make lots of optimizations with this kind of data, and they are generally considered very fast to use since locations are known and fixed.
- Stack memory
  - For data that is declared as variables within a function.
  - The location of this memory never changes for the duration of a function call; because of this compilers can optimize code so stack data is very fast to access.
- Heap memory
  - For dynamically created data
  - Data in this region may be added, moved, removed, resized, etc.
  - Because of its dynamic nature it's generally considered slower to use

Example: data & stack memory

- Example: instantiating a `struct Person` on the stack
  - The string literal `"Rust"` is read only, placed in data memory region
  - The function call `String::from` creates a `struct String` that is placed in the stack

```
struct Person {
  name: String,
}

fn main() {

  // data is on stack
  let person = Person {
    // String struct is also on stack,
    // but holds a reference to data on heap
    name: String::from("Rust"),
  };
}
```

Example: data & stack memory (2)

Heinrich Heine
Universität Düsseldorf

- Example 2: instantiating a `struct Person` on the stack using an own `new` function

```
struct Person {
  name: String,
}

impl Person {

  // Function for creating a new Person on the Stack
  fn new(name: String) -> Person {
    Person { name }
  }
}

// fn main(), see previous slide
```

Example: heap memory

Heinrich Heine
Universität Düsseldorf

- `Box` is a data structure that allows us to move our data from the stack to the heap.

```
struct Person {
  name: String,
}


fn main() {
  // instantiate Person and put it in a Box, stored in heap
  let person_boxed = Box::new( Person { name: String::from("BS-E"), } );


  println!("Name: {}", person_boxed.name);
}
```

Example: heap memory (2)

Heinrich Heine
Universität Düsseldorf

- The same example, this time with an own `new` function

```
struct Person {
  name: String,
}

impl Person {

  // Function for creating a new boxed Person stored in heap
  fn new(name: String) -> Box<Person> {
    Box::new(Person { name })
  }

}
```

Traits

- A trait is a collection of methods that a type can implement
- Traits:
  - One type can implement many traits
  - Describe behaviour but do **not** define a type
  - Traits allow defining shared behaviour between different types

```rust
trait Greet {
    fn hello(&self);
}

struct Person {
    name: String,
}
```

trait.rs

```rust
impl Greet for Person {
    fn hello(&self) {
        println!("Hello, I'm {}", self.name);
    }
}

fn main() {
    let person = Person { name: String::from("ABC"), };
    person.hello();
}
```

Traits

- Traits describe behaviour but **do not define a type**

trait.rs

```
trait Greet {
   fn hello(&self);
}

struct Person {
   name: String,
}

fn main() {
   let person = Person { name: String::from("ABC"), };
   let greet: Greet = person;
   greet.hello();
}
```

```
error[E0277]: the size for values of type `dyn Greet` cannot be known at compilation time
  --> traits.rs:20:9
   |
20 |     let greet: Greet = person;
   |         ^^^^^ doesn't have a size known at compile-time
   |
   = help: the trait `Sized` is not implemented for `dyn Greet`
   = note: all local variables must have a statically known size
help: consider borrowing here
   |
```

```rust
trait Greet {
  fn hello(&self);
}

struct Person {
  name: String,
}

fn main() {
  let person = Person { name: String::from("ABC"), };
  let greet: Greet = person;
  greet.hello();
}
```

trait.rs

Rust needs to know **at compile time** how much space any variable takes. But a trait alone doesn't tell Rust anything about the size.

Borrowed Trait Object

Heinrich Heine Universität Düsseldorf

- `&dyn Trait` → borrowed trait object

```
trait Greet {
  fn hello(&self);
}

struct Person {
  name: String,
}

fn main() {
  let person = Person { name: String::from("ABC"), };
  person.hello();

  let greet: &dyn Greet = &person;
  greet.hello();
}
```

trait.rs

- You can't use a trait as a bare type.
- You must use `dyn Greet` to make a **trait object**.
- The size for values of type `dyn Greet` cannot be known at compilation time.
  → Can't store `dyn Greet` directly by value.
- A trait object (`dyn Greet`) could be any type implementing `Greet`, so its size is unknown
  → you must use a pointer to it
- The size of the pointer is known at compile time

Owned Trait Object

- `Box<dyn Trait>` → owned trait object on the heap

trait.rs

```
trait Greet {
  fn hello(&self);
}

struct Person {
  name: String,
}

fn main() {
  let person = Person { name: String::from("ABC"), };
  person.hello();

  let greet: Box<dyn Greet> = Box::new(person);
  greet.hello();
}
```

When to use which trait object

Heinrich Heine
Universität Düsseldorf

- Use borrowed trait object `dyn &Trait` when:
  - You just want to call trait methods on it.
  - You do not need ownership, just a read-only (or &mut) view.
  - You don’t want extra heap allocation.

- Use owned trait object `Box<dyn Trait>` when:
  - You want to own “some `Greet`” without knowing its concrete type at compile time.
  - You need to store it in a `struct` or pass it around by value.
  - You might mix different types that all implement `Greet` in one collection.

Generics

- Generics let you write code once that works with **many types**
- Rust generally can infer the final type by looking at our instantiation, but if it needs help you can always be explicit using the `::<T>` operator

generics.rs

```
struct Point<T> {
  x: T,
  y: T,
}

fn main() {
  let p = Point::<i32> { x: 5, y: 10 };
  let q = Point { x: 1.2, y: 3.4 };
}
```

Generics (2)

Heinrich Heine
Universität Düsseldorf

- Implementing functions for a generic type
- Associated function:
  - Because it's associated with the type, not with a specific instance of that type.
  - Similar to static methods in Java
- `Self`: type being implemented, here `Point<T>`
- `self`: refers to the instance the method was called on.

generics2.rs

```
impl<T> Point<T> {

  // Associated function (constructor)
  fn new(x: T, y: T) -> Self {
    Point { x, y }
  }
}
```

Generics (2)

- Implementing functions for a generic type which have trait bounds

generics3.rs

```
impl<T: std::ops::AddAssign> Point<T> {

  // Associated function (constructor)
  fn new(x: T, y: T) -> Self {
    Point { x, y }
  }

  // Move the point (mutates it)
  // requires T to implement AddAssign
  fn move_by(&mut self, dx: T, dy: T) {
    self.x += dx;
    self.y += dy;
  }
}
```

Printing structs

Heinrich Heine
Universität Düsseldorf

- Derive `trait` Debug

```
#[derive(Debug)]
struct Vector {
    x: i64,
    y: i64,
}
```

```
fn main() {
    let vector = Vector { x: 42, y: 41 };
    println!("vector = {:#?}", vector);
    println!("vector = {:?}", vector);
}
```

- Pretty-printed debug output: `{:#?}` or `{:?}`

```
$ ./test
vector = Vector { x: 42, y: 41 }
vector = Vector {
    x: 42,
    y: 41,
}
```

Implementing `trait Debug`

Heinrich Heine Universität Düsseldorf

debug.rs

```
struct Vector {
  x: i64,
  y: i64,
}

impl std::fmt::Debug for Vector {
  fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
    write!(f, "Vector [0x{:x}, 0x{:x}]", self.x, self.y)
  }
}

fn main() {
  let vector = Vector { x: 42, y: 41 };
  println!("vector = {:?}", vector);
}
```

```
$ ./test
vector = Vector [0x2a, 0x29]
```

Generics & Traits

- Traits can be used to set requirements for generic types.

```
fn print_debug<T: std::fmt::Debug>(value: T) {
   println!("{:?}", value);
}
```

- Alternative, often more readable notation, using `where`

```
fn print_debug<T>(value: T)
where
   T: std::fmt::Debug,
{
   println!("{:?}", value);
}
```

Options

- Pointers are never `null`. What if you actually *want* something to be null?
- Use an `Option<T>`
- Here's the definition of `Option`, from the standard library:

```
pub enum Option<T> {
  None,
  Some(T),
}
```

- Two possible cases: the option is either `None`, or it is `Some`.
- If an Option is `Some`, the value in the `Some` variant will always be a valid value of type `T`

Option Example 1

- You can check content of Option using: `is_some()` or `is_none()`
- `assert!` Is a macro for testing or ensuring invariants
  - If it fails it will cause a panic
  - A panic happens when the program encounters a situation it can't safely continue from.

```
// This type annotation is not necessary.
let x: Option<i32> = Some(4);
assert!(x.is_some());
```

panics if `x` is `none`

Option Example 2

- `unwrap()` is a shortcut that extracts the inner value of an `Option`
- If the value is `Some`, it returns the inner value.
- If the value is `None`, it panics (crashes your program)

```
let y = x.unwrap(); // get the value out of Option
assert_eq!(y, 4);
```

panic if `x` is `none`

panic if `y` is != 4

- `expect()` does the same as `unwrap()` but panics with a given message

```
let y = x.expect("Error: x is none");
assert_eq!(y, 4);
```

Option Example 3

- Check content of option using `match`

```
let w = Some(String::from("hello"));

match w {
   Some(s) => println!("{} world!", s),
   None    => panic!("didn't expect to get here"),
}
```

`w` is `Some`, accessible in `s`

Error handling

Heinrich Heine
Universität Düsseldorf

- Most languages handle errors via one of two ways:
  - try/catch exceptions (Python, Java, JavaScript, etc.)
  - Returning a separate error value (C)

- Many times, errors are not handled properly or are tedious to handle.

- Rust tries to make error handling easier.
- It's not always smooth sailing though.

Error handling

Heinrich Heine
Universität Düsseldorf

- The idiomatic way to handle errors in Rust is via `Result<T, E>`

```
pub enum Result<T, E> {
   Ok(T),
   Err(E),
}
```

- Functions that may not complete successfully should return a `Result`.
  - If the result is `Ok`, the caller can access the returned value (of generic type `T`).
  - If the result is `Err`, additional information (of generic type `E`) about the error is returned.

- If a program cannot recover from an error,
  you can explicitly `panic!` instead of returning a `Result`.

Error handling

- Example:

```
use std::fs::File;

fn main() {
    let open_res = File::open("hello.txt");
    let f = match open_res {
        Ok(file) => file,
        Err(error) => panic!("Problem opening the file: {:?}", error),
    };
}
```

- Opening a file can fail, so `File::open` returns a `Result`.
- We check if the `open` was successful, if not, we panic

Error handling

Heinrich Heine
Universität Düsseldorf

- Matching on `Results` all the time can be tedious.
- If we know we are going to panic on an `Err`, we can use `unwrap()` instead:

```
use std::fs::File;

fn main() {
  let f = File::open("hello.txt").unwrap();
}
```

- `unwrap()` panics on error
  otherwise, it returns the data contained in the `Ok` variant.

Error handling

Heinrich Heine
Universität Düsseldorf

- Another shortcut is the `?` operator:

result.rs

```
use std::fs::File; use std::io; use std::io::Read;

fn read_file() -> Result<String, io::Error> {
   let mut f = File::open("hello.txt")?;
   let mut s = String::new();
   f.read_to_string(&mut s)?;
   Ok(s)
}
```

- If a `Result` is an `Err`, `?` causes the function to return that `Err`.
- Otherwise, `?` unwraps the `OK` variant.
- Works only if used within a function that returns `Result`

Unite type `()`

- Sometimes you want to return an `Ok` without additional information

```
fn my_func() -> Result<(), String> {
   // do something
   Ok(())
}
```

- Unite type `()` – similar to `void` in other languages

- These functions are all equivalent returning `()`

```
fn func() -> () {
   ()
}
```

```
fn func() {
   ()
}
```

```
fn func() {

}
```

Modules

- Modules serve as namespaces and are defined using the `mod` keyword.
- Code is generally private by default → can only be accessed within the current module.
- To make something visible outside a module, declare it with `pub` (for public).

```
mod inner {
  pub fn add_two(x: &mut i32) {
    *x += 2;
  }
}

fn outer() {
  let mut x = 160;
  inner::add_two(&mut x);
}
```

Modules (2)

Heinrich Heine Universität Düsseldorf

- You can move a module to a separate file, but you must import it using `mod`.
- Suppose this is in `lib.rs`:

```
mod inner;

fn outer() {
  let mut x = 160;
  inner::add_two(&mut x);
}
```

modules

- Rust will look for a file called `inner.rs` *or* `inner/mod.rs`, which should contain the contents of the module:

```
pub fn add_two(x: &mut i32) {
  *x += 2;
}
```

modules

Modules (3)

- Structuring your code
  - `inner/mod.rs`

```
pub mod more;

pub fn add_two(x: &mut i32) {
  *x += 2;
}
```

modules2

  - `inner/more.rs`

```
pub mod more;

pub fn add_threeo(x: &mut i32) {
  *x += 3;
}
```

modules2

  - `lib.rs`

```
mod inner;

fn outer() {
  let mut x = 160;
  inner::add_two(&mut x);
  inner::more::add_three(&mut x);
}

fn main() {   outer();  }
```

modules2

Modules (4)

- You can always use any (public) code by using the fully qualified name (eg. `inner::add_two`).
- `use` declarations let you "import" items so you don't have to use the full name each time:

```
mod inner;

use inner::add_two;

fn outer() {
  let mut x = 160;
  add_two(&mut x);
}
```

- You can also rename items when you import them →

```
use inner::add_two as add2;

add2(...);
```

Crates

Heinrich Heine
Universität Düsseldorf

- A crate is the biggest unit of code Rust compiles at once.
- Crates (~ packages) contain modules.

- A crate is a library or a binary/executable
- The root file is:
  - `src/lib.rs` for a library crate
  - `src/main.rs` for a binary crate

- You can publish a crate to **crates.io** or use it locally
  - https://crates.io → The Rust community's crate registry

Cargo

Heinrich Heine
Universität Düsseldorf

- Important tool for handling rust projects
- Building and running apps and tests

- Dependency management
- Building and checking code
- Running tests and examples
- Creating new crates
- Publishing packages
- Managing workspaces
- Keeping versions reproducible

Cargo dependencies: `Cargo.toml`

- You can import crates to leverage code that other people write.
- Do this by adding to your `Cargo.toml` file.
- You'll then be able to use all public items from that crate.

```
[package]
name = "hello-crate"
version = "0.1.0"
edition = "2021"

[dependencies]
rand = "0.8”
```

cargo.rs

```rust
use rand::Rng; // Import

fn main() {
   let mut rng = rand::thread_rng();
   let x: i32 = rng.gen_range(1..=6);
   println!("Gewürfelt: {x}");
}
```

Cargo dependencies: `Cargo.lock`

Heinrich Heine
Universität Düsseldorf

- Tracks all transitive dependencies
- Should not be edited
- Maintained by `cargo`
- Important command: cargo update
  - Update *all* dependencies to the newest allowed versions

Slices

- Slice: `&[T]` represents a view into a contiguous sequence of elements of type `T`
- Slices are lightweight abstractions that provide a safe and efficient way to work with a portion of a collection without needing to copy the data.
- Data is only borrowed

```
fn main() {
    let numbers = [1, 2, 3, 4, 5];

    // Take a slice of the array
    let slice = &numbers[1..4]; // Slice from [1,..4(

    // Iterate over the slice and print each element
    for &num in slice {
        println!("{}", num);
    }
}
```

Mutable Slices

- Slices can also be borrowed mutable

slices.rs

```
fn main() {
    let mut numbers = [1, 2, 3, 4, 5];

    // Take a slice of the array
    let mut_slice = &mut numbers[1..3]; // Slice from [1,..3(

    for n in mut_slice.iter_mut() {
        *n += 1;
    }

    for num in numbers {
        println!("{}", num);
    }
}
```

Does this work?

Heinrich Heine
Universität Düsseldorf

```
fn main() {
   let mut numbers = [1, 2, 3, 4, 5];

   // Take a slice of the array
   let mut_slice = &mut numbers[1..2]; // Slice from [1,..2(

   mut_slice[0] = 10;
   numbers[4]   = 0;
   mut_slice[0] = 3;
}
```

Does this work? - No

```rust
fn main() {
  let mut numbers = [1, 2, 3, 4, 5];

  // Take a slice of the array
  let mut_slice = &mut numbers[1..2]; // Slice from [1,..2(

  mut_slice[0] = 10;
  numbers[4]   = 0;
  mut_slice[0] = 3;
}
```

```
error[E0506]: cannot assign to `numbers[_]` because it is borrowed
 --> test.rs:8:3
  |
5 |   let mut_slice = &mut numbers[1..2]; // Slice from [1,..2(
  |                   ------- `numbers[_]` is borrowed here
...
8 |   numbers[4] = 0;
  |   ^^^^^^^^^^^^^^ `numbers[_]` is assigned to here but it was already borrowed
9 |   mut_slice[1] = 3;
  |   ------------ borrow later used here

error: aborting due to 1 previous error
```

Slices – Summary

Heinrich Heine
Universität Düsseldorf

- Slices are often used in Rust
- They are lightweight → data is neither copied nor moved
- Slices are safe → bounds checked as well as borrow checker

`String`

- `String`: is a growable and mutable
  - Lives in the heap
  - Is mutable and can alter its size and contents
- Variables of type `String` are fat pointers, 3 x 8 byte
  - Pointer to actual data on the heap, it points to the first character
  - Length of the string (# of characters)
  - Capacity of the string on the heap

```
// Create a new mutable empty String in the heap
let mut s = String::new();

// Push characters onto the String
s.push('h');
s.push_str("ello");
```

`&str`

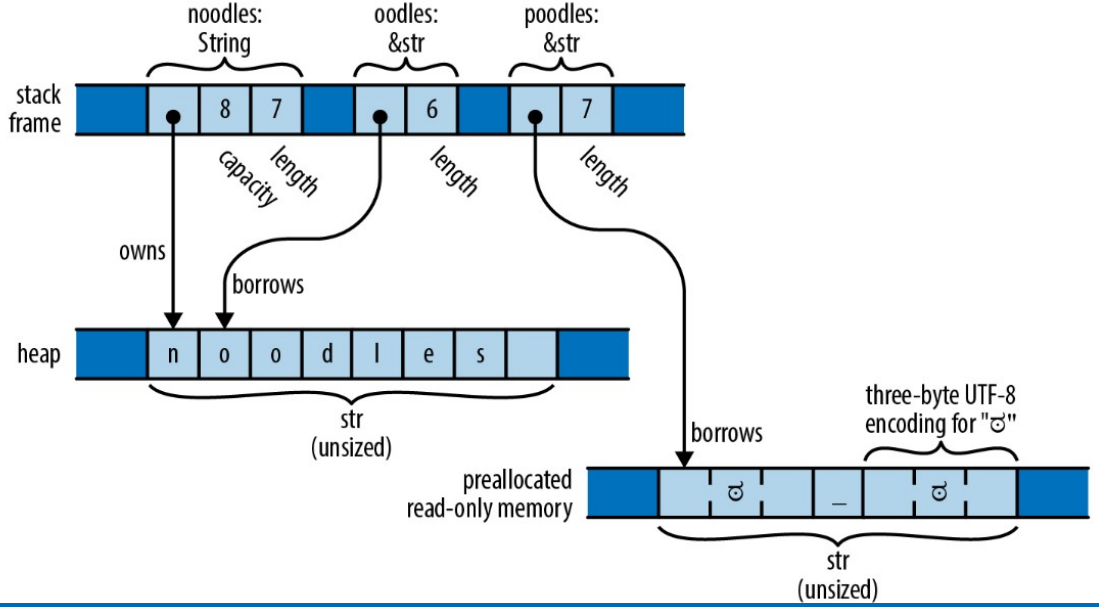
- `&str`: This denotes a reference to a string slice.
  - `&`: Indicates that it's a reference, meaning it doesn't own the data it points to. Instead, it borrows the data from another location.
  - `str`: Indicates that the data being referenced is a string slice.
- Immutable

- Variables of type `&str` are fat pointers, 2 x 8 byte
  - Pointer to the start of string data
  - Length of the string slice (# of bytes)

- String literals are stored in read-only memory
- `&str` can also borrow all or part of a `String` in heap

String vs &str

```
let noodles = "noodles".to_string();
let oodles  = &noodles[1..];
let poodles = "ಠ_ಠ"; // this is string literal
```



Thin & Fat Pointers

Heinrich Heine
Universität Düsseldorf

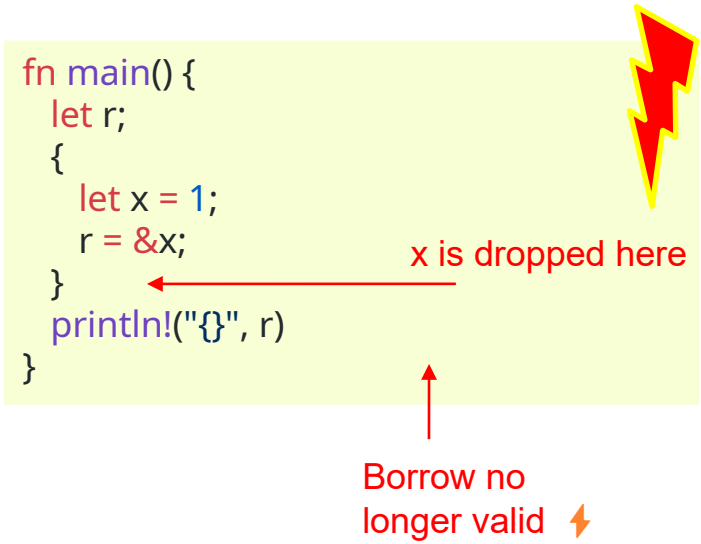
- **Thin pointer**: are used for references to **sized types**
  - These are types whose size is **known at compile time**, such as integers, structs, arrays, etc.
  - They consist of a simple memory address pointing to the data on the heap or stack
  - Examples: `&T`, `Box<T>`
- **Fat pointer**: are used for references to **unsized types**
  - These are types whose size is **not known at compile time**
  - They consist of both a memory address and metadata about the referenced data
  - Example: `&[T]`, `&str`

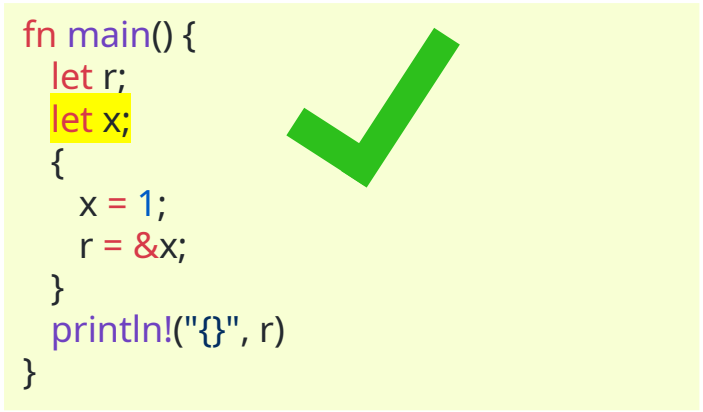
References & Lifetimes

Heinrich Heine
Universität Düsseldorf

- A reference is fundamentally just a number → start position of some bytes in memory.
- Lifetimes make sure references are always valid:
  - They prevent *dangling references* and enforce safe memory access - **at compile time**.
  - **Every reference in Rust has a lifetime** - an area of code where that reference is valid.
- Remark: lifetime annotations are only relevant for references (`&T`, `&mut T`)
  - and types that contain references.

Example: references that outlive referents





Explicit Lifetimes

Heinrich Heine
Universität Düsseldorf

- Functions sometimes must use lifetime annotations to define which inputs and outputs share the same lifetime.

lifetimes.rs

```
fn largest(x: &i32, y: &i32) -> &i32 {
  if x > y { x } else { y }
}

fn main() {
  let a = 1;
  let b = 2;
  let res = largest(&a, &b);
  println!("Largest is {}", res);
}
```

```
MBP22:lifetimes mschoett1$ rustc lifetimes.rs
error[E0106]: missing lifetime specifier
 --> lifetimes.rs:1:33
  |
1 | fn largest(x: &i32, y: &i32) -> &i32 {
  |               ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `x` or `y`
help: consider introducing a named lifetime parameter
  |
1 | fn largest<'a>(x: &'a i32, y: &'a i32) -> &'a i32 {
  |           ++++     ++          ++          ++

error: aborting due to 1 previous error
```

lifetimes.rs

```rust
fn largest(x: &i32, y: &i32) -> &i32 {
  if x > y { x } else { y }
}

fn main() {
  let a = 1;
  let b = 2;
  let res = largest(&a, &b);
  println!("Largest is {}", res);
}
```

Explicit Lifetimes

- Lifetime specifiers always start with a `'` (e.g. `'a`, `'b`, `'c`, ...)
- Corrected example:

```
fn largest<'a>(x: &'a i32, y: &'a i32) -> &'a i32 {
  if x > y { x } else { y }
}
```

  - The `<'a>` means: function works for any lifetime `'a` (syntax similar to `<T>`)
  - The `&'a` before the parameters mean: both references have the same lifetime
  - The `&'a` before the return type means: the returned lives as long as both parameters

Lifetime Elision[1] Rules

Heinrich Heine
Universität Düsseldorf

- Rust has three simple rules for functions involving references.
  1. Each parameter that is a reference gets its own lifetime parameter.
  2. If there is exactly one input lifetime, it's assigned to all output references.
  3. If there are multiple input lifetimes and the output is a reference, the compiler needs explicit annotations (no guessing).
- This example is OK without explicit lifetime annotations

```
fn foo(x: &i32) -> &i32 {
  x
}
```

- The Rust compiler automatically adds annotations, if possible:

```
fn foo<'a>(x: &'a i32) -> &'a i32 {
  x
}
```

[1] Elision = Weglassen

Lifetimes in data types

Heinrich Heine
Universität Düsseldorf

- Similarly to functions, data types can be parameterized with lifetime specifiers of its members.
- Rust validates that the containing data structure of the references never lasts longer than the owners its references point to.
- We can't have structs running around with references pointing to nothing!

Lifetimes in data types

- Example:

```
struct Config {
   ...
}

struct App {
   config: &Config
}
```

```
struct Config {
   ...
}

struct App<'a> {
   config: &'a Config
}
```

- We need to tell the compiler, that config, which is of type &Config, has the same lifetime as App.

Multiple lifetimes in data types

- Example:

```
struct Point<'a> {
   x: &'a i32,
   y: &'a i32
}
```

```
fn main() {
   let x = 3;
   let r;
   {
      let y = 4;
      let point = Point { x: &x, y: &y };
      r = point.x
   }
   println!("{}", r);
}
```

- After this update the code works:

```
struct Point<'a,'b > {
   x: &'a i32,
   y: &'b i32
}
```

Static lifetime

Heinrich Heine
Universität Düsseldorf

- The `'static` lifetime is a special lifetime that represents the entire duration of the program.
- Any reference with a `'static` lifetime can be used anywhere without worrying about its scope.
- Should only be used if necessary → `'static` resources will never drop.

- Example:

```
// string literals have a 'static lifetime, so 'static is here optional
const MSG: &'static str = "Hello World!";

fn main() {
   println!("msg = {}", MSG);
}
```

Final remarks on lifetime annotations

Heinrich Heine
Universität Düsseldorf

- Lifetime annotations are only relevant for references (`&T`, `&mut T`) and types that contain references.

- Compiler does lifetime annotations automatically whenever possible otherwise, it will ask for explicit lifetime annotations

4.3 Unsafe stuff

Heinrich Heine
Universität Düsseldorf

- Rust's compiler enforces memory and thread safety
- Some low-level operations in an OS require direct memory access.
  - These code sections need to be marked as `unsafe`
  - Should be used only if functionality can not be expressed in clean Rust
  - The programmer is taking explicit responsibility
- Things you can only do in `unsafe`
  - 1. Dereference raw pointers: direct memory access, memory mapped IO
  - 2. Call unsafe functions: syscalls, inline assembly
  - 3. Access or modify mutable static variables: kernel globals

`unsafe` blocks & functions

Heinrich Heine
Universität Düsseldorf

- `unsafe` means “I promise the safety invariants hold here.”
- Example: `unsafe` block

```
pub fn write_u8(addr: *mut u8, val: u8) {
  unsafe { core::ptr::write_volatile(addr, val); }
}
```

- Example: `unsafe` function (can only be called inside an `unsafe` block)

```
pub unsafe fn write_u8(addr: *mut u8, val: u8) {
  unsafe { core::ptr::write_volatile(addr, val); }
}
```

- Remarks:
  - **volatile** prevents optimization away of I/O.
  - `core::ptr` is a raw pointer, see next slides

Raw Pointers

Heinrich Heine
Universität Düsseldorf

- References can be converted into a more primitive type called a **raw pointer**
  - Much like a number, it can be copied and moved around with little restriction.
  - Rust makes no assurances of the validity of the memory location it points to.

- Two kinds of raw pointers exist:
  - `*const T` - A raw pointer to data of type `T` that should never change.
  - `*mut T` - A raw pointer to data of type `T` that can change.

Raw Pointers: Example 1

Heinrich Heine
Universität Düsseldorf

- **Accessing a memory-mapped register**

```
pub fn uart_write(ch: u8) {
    const UART0: *mut u8 = 0x1000_0000 as *mut u8;
    unsafe { core::ptr::write_volatile(UART0, ch); }
}
```

- The pointer `UART0` is constant (`const`), so no accidental mutation of the address.
- Writing to the address is `unsafe`
- `volatile` prevents optimization away of I/O.

Raw Pointers: Example 2

- **Mutable raw pointer from a Rust reference**

```
fn example() {
   let mut x = 10;
   let ptr = &mut x as *mut i32;
   unsafe { *ptr += 1; }
   println!("{}", x);
}
```

- As `ptr` is mutable we can do pointer arithmetic, which is obviously unsafe

Raw Pointers: Example 3

Heinrich Heine
Universität Düsseldorf

```
#[repr(C)]    // C-compatible layout
struct DevRegisters {
   r1: u32,
   r2: u32,
}

const DEV_BASE: *mut DevRegisters = 0x1000 as *mut DevRegisters;

unsafe fn get_dev_regs() -> &'static mut DevRegisters {
   &mut *DEV_BASE
}

fn main() {
   let dev_regs;
   unsafe { dev_regs = get_dev_regs(); }
   dev_regs.r1 = 0x41; // write to register r1, safe, using reference
}
```

References vs Raw Pointers

Heinrich Heine
Universität Düsseldorf

- Rust reference:
  - Similar to a pointer in C in terms of usage
  - But with much more compile time restrictions on
    how it can be stored and moved around to other functions.

- Rust raw pointer:
  - Also similar to a pointer in C
  - Represents a number that can be copied or passed around,
    and even turned into numerical types
    where it can be modified as a number to do pointer math.

4.4 Summary

Heinrich Heine
Universität Düsseldorf

- We covered advanced language concepts of Rust
- Note there are no classes with code inheritance
- Unsafe code is needed in OS and driver development
  - Use unsafe only when absolutely necessary
  - Allows violating Rust safety, so should be used carefully

4.5 Credits & further information

Heinrich Heine
Universität Düsseldorf

- Many slides from Rohan Kumar, Rahul Kumar, and Edward Zeng
  - Used in the course CS 162: Operating Systems and Systems Programming, Prof. John Kubiatowicz at Berkely University, USA
- Interactive Rust book: https://rust-book.cs.brown.edu
- Libraries: https://lib.rs
  - 205672 Rust libraries and applications.